Phase 1 Report: Responsibility Mapping & Asset Definition

Project: Securing a 3-Tier Web Application in the Cloud (Unit 1)

Name: Sumit Mhalaskar

PRN: 2124UCSM1065

Program: B.Tech Cyber Security, Sanjivani University

1. Introduction and Objectives

Cloud security is built on one idea: security is shared between the cloud provider and the customer. Providers secure the cloud itself (data centres, hardware, hypervisors, core services). Customers secure what they put in the cloud (identities, configuration, data, application code). Most real-world cloud breaches come from the customer's side of that line, usually through misconfiguration, weak identity controls or exposed data.

This report covers Phase 1 of the project. Its objectives are:

- Design the architecture of a 3-tier web application (Web, App and Database tiers).
- Define and classify the assets that make up the application.
- Build a Shared Responsibility Matrix tailored to this application, stating which security controls belong to the cloud provider and which belong to us.
- Identify the customer-side control areas that Phases 2 onward must implement.

2. Application Overview and Assumptions

The sample application is an **online book store**: customers browse books, create accounts, place orders and pay online. It handles personal data (names, addresses, emails), credentials and order history, so it is a realistic target for security design.

Assumptions

Item	Choice
Cloud provider	Amazon Web Services (AWS). The model applies equally to Azure and GCP with renamed services.
Service model	Mixed IaaS and PaaS: virtual machines or containers for Web and App tiers, a managed relational database for the DB tier.

Item	Choice
Region	Single region, two Availability Zones for resilience.
Payments	Handled by a third-party payment gateway; card numbers are never stored by the application.
Users	Anonymous visitors, registered customers, administrators.

3. Three-Tier Architecture Design

3.1 Tier Summary

Tier	Role	Components	AWS Service	Network Placement	Exposure
Web (Presentation)	Serves the user interface and terminates HTTPS	React frontend, reverse proxy	CloudFront, Application Load Balancer, S3 (static assets), EC2/ECS web instances	Public subnets	Internet-facing
App (Logic)	Business logic, authentication, APIs	Node.js REST API (orders, catalogue, users)	EC2/ECS app instances behind internal load balancer	Private app subnets	Reachable only from Web tier
DB (Data)	Persistent storage	Relational database for users, books, orders	Amazon RDS (MySQL/PostgreSQL), Multi-AZ	Private DB subnets	Reachable only from App tier

3.2 Request Flow

1. The user's browser connects over HTTPS to CloudFront, which forwards to the Application Load Balancer.
2. The load balancer routes traffic to the web instances, which serve the React app and proxy API calls.
3. API calls pass to the internal load balancer and then to the Node.js app instances.
4. The app tier authenticates the request, applies business rules and queries the database on port 3306/5432.
5. The response travels back along the same path. Outbound calls (payment gateway, email) leave through a NAT gateway.

3.3 Network Segmentation

Zone	Subnet Type	Allowed Inbound	Allowed Outbound
Edge	Public	443 from internet	To app tier only
Web	Public/private	443 from load balancer	To internal load balancer
App	Private	App port from web tier security group	To DB tier, NAT gateway
DB	Private, no internet route	DB port from app tier security group	None

Security groups act as stateful firewalls per tier, and network ACLs give a second, stateless layer at subnet level. Administrative access uses a bastion host or AWS Systems Manager Session Manager, never open SSH from the internet.

4. Asset Definition

Assets are classified by confidentiality, integrity and availability (CIA) impact: High (H), Medium (M) or Low (L).

ID	Asset	Tier	Type	Classification	C	I	A
A1	React frontend code and static assets	Web	Software/data	Public	L	M	M
A2	Web server instances and OS	Web	Compute	Internal	M	M	M
A3	TLS certificates and private keys	Web	Credential	Confidential	H	H	M
A4	Node.js application code and dependencies	App	Software	Internal	M	H	M
A5	App server instances and OS	App	Compute	Internal	M	H	H
A6	API keys and secrets (payment gateway, DB credentials)	App	Credential	Restricted	H	H	M
A7	User accounts and password hashes	DB	Data	Restricted	H	H	M
A8	Customer personal data (name, address, email, phone)	DB	Data	Confidential	H	H	M

ID	Asset	Tier	Type	Classification	C	I	A
A9	Order and transaction records	DB	Data	Confidential	M	H	H
A10	Database backups and snapshots	DB	Data	Confidential	H	H	M
A11	IAM users, roles and policies	Cross-cutting	Identity	Restricted	H	H	H
A12	Logs and audit trails (CloudTrail, application logs)	Cross-cutting	Data	Internal	M	H	M
A13	VPC, subnets, security groups, routing	Cross-cutting	Network config	Internal	M	H	H
A14	CI/CD pipeline and source repository	Cross-cutting	Software/process	Internal	M	H	M

The highest-value assets are A6, A7, A8 and A11: compromise of credentials, customer data or identity configuration would have the largest impact, and all four sit on the customer side of the responsibility line.

5. The Shared Responsibility Model

Responsibility shifts toward the provider as the service becomes more managed.

Layer	IaaS (e.g. EC2)	PaaS/Managed (e.g. RDS)	SaaS
Physical data centre, hardware, global network	Provider	Provider	Provider
Hypervisor and virtualisation	Provider	Provider	Provider
Operating system patching	**Customer**	Provider	Provider
Database engine patching	Customer	Provider (customer sets maintenance window)	Provider
Network and firewall configuration	**Customer**	**Customer**	Provider
Application code and logic	**Customer**	**Customer**	Provider
Identity and access management	**Customer**	**Customer**	**Customer**

Layer	IaaS (e.g. EC2)	PaaS/Managed (e.g. RDS)	SaaS
Data classification, protection, encryption choices	**Customer**	**Customer**	**Customer**

In our design, the Web and App tiers use IaaS (we own the OS upward) and the DB tier uses managed RDS (the provider owns the OS and engine patching, we own configuration, access and data).

6. Shared Responsibility Matrix for the Application

Key: P = Provider, C = Customer (us), S = Shared (each owns a part, explained in the notes).

6.1 Cross-Cutting and Infrastructure Controls

Control Area	Specific Control	P	C	S	Notes
Physical security	Data centre access, power, cooling, environmental controls	X			AWS responsibility; validated by SOC 2 and ISO 27001 reports
Hardware	Servers, storage, hardware lifecycle and disposal	X			Provider-managed, including media sanitisation
Global network	Backbone, DDoS baseline protection (Shield Standard)	X			Provider; advanced protection is optional and customer-enabled
Virtualisation	Hypervisor isolation between tenants	X			Provider
Managed service internals	RDS host OS, engine installation, Multi-AZ failover mechanism	X			Provider
Identity and access (IAM)	User/role design, least privilege, MFA, root account protection, key rotation		X		Customer entirely; the most common source of breaches
Account governance	Billing alerts, organisation guardrails, region restrictions		X		Customer
Logging and audit	Enabling CloudTrail, VPC Flow Logs, centralised log storage and retention			X	Provider supplies the service; customer must enable, protect and review logs
Monitoring and alerting	CloudWatch alarms, GuardDuty, incident detection			X	Provider supplies tools; customer configures and responds

Control Area	Specific Control	P	C	S	Notes
Incident response	Response plan, roles, containment and recovery		X		Customer; provider handles incidents on its own infrastructure
Compliance	Mapping to GDPR, India DPDP Act, PCI DSS scope			X	Provider certifies its infrastructure; customer is responsible for compliant use
Backup and disaster recovery	Backup schedule, restore testing, cross-AZ design			X	Provider offers snapshots and multi-AZ; customer defines and tests strategy
Supply chain / CI-CD	Pipeline security, dependency scanning, code signing		X		Customer

6.2 Web Tier (Public-Facing)

Control	P	C	S	Notes
Physical host and hypervisor security	X			Provider
OS hardening and patching of web instances		X		IaaS: customer owns the OS
Web server configuration (disable unused modules, secure headers)		X		Customer
TLS certificate issuance and renewal (ACM)			X	Provider offers ACM; customer chooses protocol versions, ciphers and applies HSTS
Security groups and NACLs for web subnet		X		Customer
Web Application Firewall rules (AWS WAF)			X	Provider supplies the service; customer writes and tunes rules (OWASP Top 10)
DDoS protection beyond baseline			X	Shield Advanced is optional and customer-enabled
Frontend code security (XSS, CSP, dependency hygiene)		X		Customer

Control	P	C	S	Notes
S3 bucket policies for static assets (block public write, least privilege)		X		Customer; misconfiguration is a leading cause of exposure
Malware and endpoint protection on web instances		X		Customer

6.3 Application Tier (Business Logic)

Control	P	C	S	Notes
Host and hypervisor security	X			Provider
OS and runtime patching (Node.js, npm packages)		X		Customer
Authentication and session management (password hashing, JWT handling, MFA for admins)		X		Customer
Authorisation and role-based access control in the API		X		Customer
Input validation and output encoding (injection, XSS prevention)		X		Customer
API rate limiting and abuse protection		X		Customer (can use API Gateway or WAF features)
Secrets management (AWS Secrets Manager, no hard-coded keys)			X	Provider supplies the vault; customer decides what to store and rotates secrets
Instance IAM roles and permissions		X		Customer: least privilege only
Internal load balancer and security group rules		X		Customer
Vulnerability scanning and penetration testing		X		Customer (pen-test within AWS policy)
Application logging (avoid logging sensitive data)		X		Customer

6.4 Database Tier (Data)

Control	P	C	S	Notes
Physical storage security and disk-level isolation	X			Provider
Host OS and DB engine patching (RDS)	X			Provider applies patches; customer chooses the maintenance window and engine version
Network isolation (private subnet, no public access)		X		Customer configures subnet group and security group
Database authentication and user privileges		X		Customer: least-privilege DB users, no shared admin account
Encryption at rest (KMS)			X	Provider supplies encryption; customer must enable it and manage keys
Encryption in transit (force TLS connections)			X	Provider supports TLS; customer enforces it
Backups and snapshots			X	Provider runs automated backups; customer sets retention, encrypts and tests restores
High availability (Multi-AZ)			X	Provider executes failover; customer chooses to enable it
Data classification and retention		X		Customer
Data privacy and compliance (PII handling, deletion requests)		X		Customer is the data controller
Database activity auditing			X	Provider offers audit logging; customer enables and reviews it

7. Responsibility Summary and Risk Observations

Tier	Mostly Provider	Mostly Customer	Shared
Web	Physical, hypervisor	OS, web config, frontend code, firewalling	TLS, WAF, DDoS

Tier	Mostly Provider	Mostly Customer	Shared
App	Physical, hypervisor	Nearly everything above the hypervisor	Secrets management
DB	Host OS, engine patching, hardware	Access control, network isolation, data governance	Encryption, backup, auditing

Key observations

1. **The App tier carries the largest customer burden.** Everything above the hypervisor is ours, so it needs the most controls.
2. **Shared does not mean automatic.** Features such as encryption at rest, Multi-AZ, CloudTrail and WAF exist but are not secure until we enable and configure them.
3. **Identity is the common thread.** IAM misconfiguration affects all three tiers and is entirely our responsibility.
4. **Data is always ours.** Regardless of service model, classification, privacy compliance and protection of customer data stay with the customer.

8. Mapping Threats to Responsibility

Threat	Affected Tier	Who Owns the Control	Primary Mitigation
SQL injection	App, DB	Customer	Parameterised queries, input validation, least-privilege DB user
Cross-site scripting	Web, App	Customer	Output encoding, Content Security Policy
Credential stuffing and brute force	App	Customer	MFA, rate limiting, account lockout
Exposed S3 bucket or database	Web, DB	Customer	Block public access, private subnets, regular config audits
Unpatched OS or library	Web, App	Customer	Patch management, dependency scanning
Leaked access keys	All	Customer	IAM roles instead of keys, Secrets Manager, rotation
Volumetric DDoS	Web	Shared	Shield, CloudFront, WAF rate rules

Threat	Affected Tier	Who Owns the Control	Primary Mitigation
Hypervisor escape or hardware compromise	All	Provider	Provider controls and certifications
Data loss or ransomware	DB	Shared	Encrypted, tested backups, cross-region copy

9. Conclusion and Next Steps

This phase defined a 3-tier architecture with segmented Web, App and DB layers, catalogued fourteen assets with CIA ratings, and produced a Shared Responsibility Matrix specific to this application. The main finding is that the provider secures the foundation, but nearly every control that determines whether this application is actually secure (identity, configuration, code, data protection and monitoring) is ours to implement.

The matrix will guide later phases: the customer-owned and shared rows become the checklist for identity and access design, network and data protection controls, and monitoring and incident response.

10. References

- AWS, *Shared Responsibility Model*, aws.amazon.com/compliance/shared-responsibility-model
- AWS, *Well-Architected Framework: Security Pillar*
- NIST SP 800-145, *The NIST Definition of Cloud Computing*
- Cloud Security Alliance, *Security Guidance for Critical Areas of Focus in Cloud Computing*
- OWASP, *Top 10 Web Application Security Risks*